

LLMSheriff: A Hybrid Framework for Intent-State Inference in Autonomous AI Agents

Rajendra Kumar Shaw
Kolkata, India
rajendra250601@gmail.com

Abstract—Current observability systems expose execution events but do not explicitly indicate whether an autonomous agent is still making meaningful progress toward its goal. As a result, developers struggle to distinguish productive execution, temporary recovery, prolonged waiting, and silent failure. This paper presents LLMSheriff, an exploratory prototype that investigates behavioral state inference from execution traces using hybrid rule-based and LLM analysis. Eight coarse states—Planning, Executing, Waiting, Recovering, Stalled, Abandoned, Completed, and Failed—are inferred in parallel. Analyzer agreement is treated as an interpretable confidence indicator; disagreement is surfaced as a diagnostic cue for human inspection rather than forced resolution. On a controlled synthetic corpus of 100 labelled traces (primary hybrid run on the AWS programme environment), the rule engine reaches 96% accuracy and the LLM judge 66%; when both agree ($n=64$), accuracy relative to synthetic labels is 98.4%. Hybrid value is framed as informative diagnosis via agreement/disagreement triage rather than as unconditional accuracy gains over tuned rules. These results demonstrate feasibility under controlled conditions; they do not establish generalisation to production agent systems. The contribution is a reproducible hybrid approach to behavioral diagnosis above conventional observability, not a claim of universal superiority over existing platforms.

Index Terms—agentic AI, execution monitoring, intent inference, goal satisfaction, behavioral observability

I. Introduction

Autonomous AI agents orchestrate multi-step workflows involving tool calls, retries, external dependencies, and long idle periods. Standard observability outputs—logs, latency metrics, retry counts, and exit codes—record what happened during execution. They rarely answer whether the agent is still making meaningful progress toward its assigned goal.

This gap matters in practice. An agent may run without errors while making no forward progress: repeating the same search, polling indefinitely, or drifting after an early failure. Conversely, an agent may encounter setbacks and still recover by adopting a new strategy. A monitoring layer that only detects hard failures cannot separate productive execution, temporary recovery, prolonged waiting, and silent failure—leaving developers uncertain whether to wait, intervene, or declare success.

Existing observability platforms emphasize execution traces, metrics, and debugging views. This paper investigates a complementary question: behavioral state inference for runtime diagnosis. LLMSheriff is a research

prototype that maps traces to coarse behavioral states using a hybrid of deterministic rules and an LLM behavioral judge. A central design choice is that the system does not attempt to recover an agent’s true internal intentions. Instead, it infers behavioral intent from externally observable events—a weaker but tractable formulation. Critically, when the two analyzers disagree, the conflict is preserved as an uncertainty signal rather than suppressed.

a) **Research question.**: Can hybrid rule-based and LLM-based behavioral inference improve the diagnosis of autonomous AI agent execution traces, and can agreement/disagreement serve as an interpretable confidence signal?

b) **Contributions.**:

- A hybrid inference pipeline that produces explainable behavioral states from traces, with evidence summaries and intervention recommendations.
- An explicit agreement/disagreement mechanism in which analyzer conflict is treated as a first-class diagnostic output for human review (the primary empirical focus).
- A reproducible synthetic evaluation that tests the hypotheses below under controlled conditions, with transparent limitations.

c) **Hypotheses.**:

- H1: A hybrid framework provides more informative behavioral diagnosis than either analyzer alone by exposing complementary agreement and disagreement patterns (rather than by unconditionally beating rule-only label accuracy).
- H2: Agreement between rule-based and LLM inference serves as an interpretable confidence indicator, while disagreement identifies a focused subset of traces that warrant human inspection.

We do not claim that LLMSheriff is the best agent observability system. We claim that this behavioral inference approach is useful and promising under the evaluation performed.

II. Related Work

A. AI Observability and Tracing

Platforms such as LangSmith [6], Langfuse [7], and AgentOps [8] provide trace storage, latency analysis, token accounting, and debugging views for LLM applications.

OpenTelemetry [5] standardises distributed tracing across services. These tools excel at reconstructing execution paths and surfacing failures. They do not, by default, infer goal-level behavioral states for wait-versus-intervene decisions.

B. Agent Monitoring and Oversight

Recent work on LLM agents emphasises tool-using workflows [11] and self-reflective repair loops [12]. Oversight research often targets unsafe tool calls, hallucinated actions, policy violations, and structured provenance for accountability. Complementary evaluation research uses LLMs as judges of response quality [13]. LLMSheriff differs by targeting progress semantics on execution traces: whether behaviour still appears goal-directed rather than merely error-free, policy-compliant, or locally high-quality. An agent that loops safely can still fail its user.

C. Goal Recognition and Runtime Verification

Goal-oriented requirements engineering treats systems in terms of goals and obstacles to satisfaction [1], [2]. Plan and goal recognition infer intentions from observed behaviour [3]. Runtime verification checks formal properties during execution [4]. LLMSheriff applies a lightweight, pragmatic variant of these ideas to LLM-agent traces: coarse state classification with hybrid symbolic and LLM judgement, rather than full plan reconstruction or temporal-logic specifications.

D. Behavioral Science of AI Agents

A growing research direction treats AI agents as behavioral subjects rather than only as models to be inspected from the inside [9]. Parallel work builds experimental frameworks that probe agent choice under controlled manipulations of attributes and persuasive cues (e.g. ABxLab in consumer-choice settings) [10]. These efforts ask how agents act, adapt, and decide in context—moving beyond task-success metrics alone.

LLMSheriff addresses a complementary runtime question. Existing behavioral-science work largely focuses on understanding or experimentally probing agent behavior. We instead investigate runtime behavioral diagnosis from execution traces: inferring coarse progress states and using hybrid rule+LLM agreement/disagreement as an operational confidence and triage mechanism for human operators.

E. Positioning

Observability stacks answer what happened; behavioral experiments answer how agents choose under interventions. Our work focuses on what behavioral state the agent appears to be in now, for wait-versus-intervene decisions, with agreement/disagreement as an interpretable confidence signal. LLMSheriff should be viewed as a runtime diagnosis / interpretation layer within this emerging landscape—above conventional observability, and complementary to choice-oriented behavioral benchmarks.

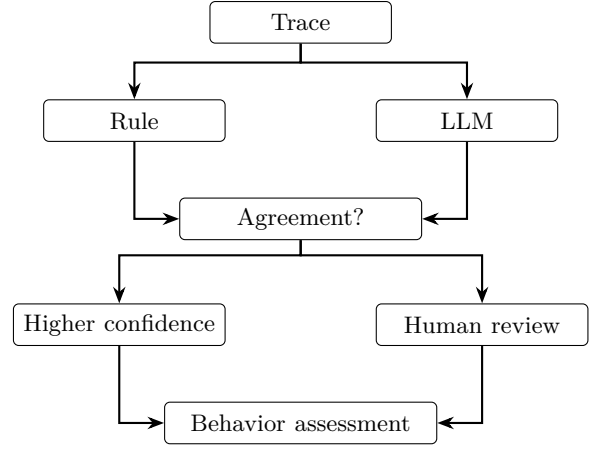


Fig. 1. Conceptual flow: dual inference uses agreement as a confidence cue and disagreement as a triage signal for human inspection.

III. Method

A. Conceptual Overview

Figure 1 summarises the idea before implementation detail. A trace is scored by rules and by an LLM in parallel; agreement supports higher-confidence acceptance, while disagreement routes the case to a human along with both predictions.

B. Architecture

Figure 2 shows the end-to-end pipeline. Execution traces undergo feature extraction, pass through two parallel analyzers, and feed a behavior interpretation layer that produces a structured assessment for human operators.

C. Feature Extraction

Each event contains a timestamp, step label, action string, duration, and status. Twelve features are computed: runtime, average latency, LLM/tool/retry/failure counts, longest repeated-action streak, a progress score (success rate penalised for non-waiting repetition, capped at 1.0), waiting-action count/ratio (poll/status-check vocabulary), and maximum/mean idle gaps. Idle-gap features separate tight work loops (Stalled) from long-idle drift (Abandoned).

D. Rule Engine and LLM Judge

Rules are fast, deterministic, and explainable. They encode thresholds for failures, waiting polls, tight loops, recovery after errors, and abandonment-like idle drift. The LLM judge (NVIDIA Nemotron [14]) receives the full trace and metrics and returns a structured state prediction; a local heuristic fallback is used if the API fails. Rules struggle on borderline cases; the LLM can reason about action semantics but is slower, API-dependent, and less stable.

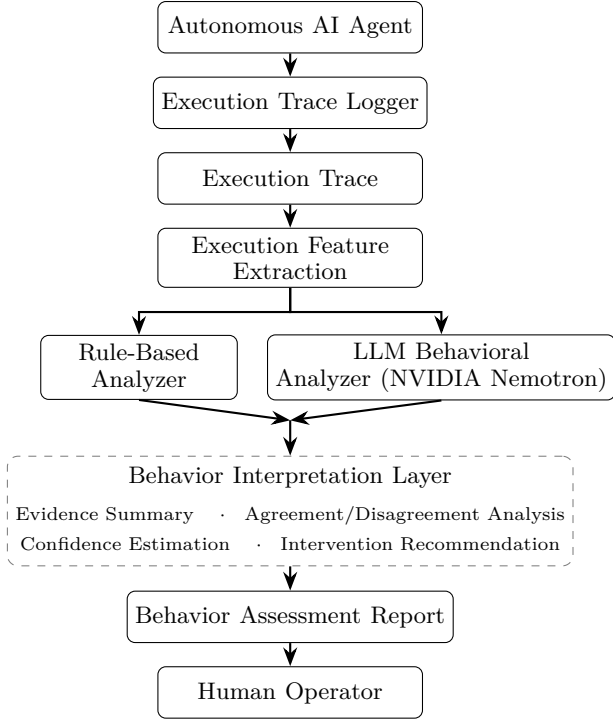


Fig. 2. LLMSheriff pipeline. Parallel rule and LLM analyzers feed a behavior interpretation layer that treats agreement as confidence and disagreement as a diagnostic signal for human operators.

E. Agreement Logic

If both analyzers predict the same state, the system reports higher confidence. If they disagree (e.g., rule = Waiting, LLM = Stalled), LLMSheriff does not silently pick a winner. It surfaces low confidence and recommends human inspection. This disagreement-as-signal design is central to the contribution.

F. State Vocabulary

We use eight coarse behavioral states. Definitions are operational (observable from traces), not claims about hidden mental states:

- Planning: the agent is organising next steps before substantial tool work.
- Executing: the agent is making varied, goal-directed progress.
- Waiting: the agent is polling or idle on an external dependency rather than doing productive local work.
- Recovering: after failures or setbacks, the agent changes strategy and continues toward the goal.
- Stalled: the agent repeats tight, low-diversity work without progressing (busy but unproductive).
- Abandoned: long idle gaps and drift suggest the goal is no longer being pursued.
- Completed: the trace indicates successful goal termination.
- Failed: the trace indicates explicit unsuccessful termination.

Distinguishing Waiting from Stalled and Abandoned is central: waiting may be legitimate; stalled and abandoned motivate intervention.

G. Implementation Note

A FastAPI backend and Next.js dashboard visualise dual predictions, evidence, disagreement, and intervention recommendations. The dashboard supports exploration; it is not the research contribution. Reproducible evaluation scripts live in `backend/scripts/`.

IV. Evaluation

A. Dataset and Protocol

Why synthetic. No publicly available benchmark exists for agent behavioral state inference with wait-versus-intervene labels. Collecting and independently annotating production traces at scale was out of scope for this programme sprint. We therefore built a controlled synthetic corpus to test feasibility of H1/H2 reproducibly.

Construction. The generator (`backend/scripts/synthetic_traces.py`) emits 100 traces with a fixed seed (default 42), balanced at 20 traces for each of five evaluated classes: Completed, Recovering, Waiting, Stalled, and Abandoned. Planning/Executing/Failed appear as intermediate event structure inside generators but are not separate evaluation labels. Each event has timestamp, step, action, duration, and status. Class semantics are encoded explicitly:

- Completed: short planning then diverse successful tool/LLM steps.
- Recovering: early failed tool calls, then a changed successful strategy.
- Waiting: deployment followed by repeated “Poll CI status” actions with moderate gaps.
- Stalled: many repetitions of the same search action with short gaps (busy loop).
- Abandoned: early work then repeated idle “Determine next action” steps with long gaps (35–50s).

Labels. Gold labels equal the generator class identity. They are not independent human annotations. Rules were hand-tuned with knowledge of these generator families, so high rule accuracy measures feasibility on controlled trajectories, not production generalisation. Evaluation scripts live in `backend/scripts/`.

B. Metrics

We report accuracy and macro-F1 against synthetic gold labels, agreement rate, and conditional accuracy when analyzers agree or disagree. For H2 we additionally report an inspection budget: the fraction of traces routed to human review under the policy “review iff analyzers disagree,” and how many rule errors fall inside that set.

C. Evidence for H1 (Informative Hybrid Diagnosis)

Table I shows rule-only performance after encoding waiting and idle-gap features: 96% accuracy, macro-F1 0.978. Residual errors are concentrated on borderline Stalled traces (Table II).

TABLE I
Rule-engine performance on 100 synthetic traces

Class	Prec.	Recall	F1	n
Completed	1.00	1.00	1.00	20
Recovering	1.00	1.00	1.00	20
Waiting	1.00	1.00	1.00	20
Stalled	1.00	0.80	0.89	20
Abandoned	1.00	1.00	1.00	20
Overall	Accuracy 0.96 / Macro-F1 0.978			100

TABLE II
Selected confusion patterns (ground truth $\rightarrow$ prediction)

Pattern	Count
Stalled $\rightarrow$ Executing	4

Table III compares rule-only, LLM-only, and the agreement subset for the primary hybrid evaluation (E04). The LLM alone reaches 66% accuracy (macro-F1 0.642; 86/100 successful API parses, with 13 timeout/error fallbacks and 1 parse fallback). Under the revised H1, we do not claim that a fused hybrid label beats rule-only accuracy: rules already dominate (0.96 vs 0.66). Support for H1 is instead diagnostic: dual inference exposes complementary patterns—agreement isolates high-confidence cases, and disagreement surfaces the LLM’s three Stalled catches that rules miss (Section IV-D). A single analyzer cannot provide that agreement signal.

TABLE III
Rule vs LLM on 100 synthetic traces (E04). LLM includes 14/100 API fallbacks; rules are hand-tuned on this corpus.

Analyzer	Acc.	Macro-F1
Rule engine	0.96	0.978
LLM judge	0.66	0.642
Agree subset ($n=64$)	0.984	—

D. Evidence for H2 (Agreement as Confidence)

Table IV is the primary H2 result. Agreement occurred on 64/100 traces (64%). When analyzers agreed, accuracy relative to synthetic gold was 98.4% (63/64); the single agree-but-wrong case was a Stalled trace where both predicted Executing after an LLM parse fallback. When they disagreed ($n=36$), the rule engine was correct on 91.7% of cases and the LLM on 8.3%.

TABLE IV
Agreement as a confidence signal (H2; E04)

Subset	n	Acc. vs gold
Agree	64	0.984
Disagree (rule)	36	0.917
Disagree (LLM)	36	0.083

Operationalising human inspection. We define the review set as all disagreement traces. Under this policy (Table V):

- Inspection budget: 36% of traces are flagged; 64% are agreement candidates for lower-touch handling.
- Error concentration: 3 of 4 rule errors (75%) fall inside the review set; the remaining rule error is the agree-wrong fallback case.
- Complementary catches: all 3 LLM-only-correct cases are disagreements (Stalled recovered while rules predicted Executing).
- Ambiguity isolation: disagreements also include systematic LLM failure modes (Abandoned $\rightarrow$ Stalled, Completed $\rightarrow$ Executing), so review is not assumed to prefer the LLM label—only to attend to conflict.

TABLE V
Disagreement as inspection triage (E04)

Quantity	Value
Auto-triaged (agree)	64%
Acc. on agree subset	0.984
Flagged for review	36%
Rule errors in review set	3/4
LLM-only corrects in review set	3/3

Disagreements concentrated on Abandoned $\rightarrow$ Stalled by the LLM ($n=20$), Completed $\rightarrow$ Executing ($n=11$), Stalled rule misses recovered by the LLM ($n=3$), and Waiting $\rightarrow$ Executing ($n=2$) (Table VI). Thus disagreement improves diagnosis by concentrating residual risk and complementary corrections into a minority review queue, not by magically resolving labels.

TABLE VI
Disagreement patterns by true label (E04)

True	n	Rule	LLM
Abandoned	20	Abandoned	Stalled
Completed	11	Completed	Executing
Stalled	3	Executing	Stalled
Waiting	2	Waiting	Executing

Correctness breakdown: both correct 63; rule-only 33; LLM-only 3; both wrong 1. These results support H2 on this dataset: agreement is a strong confidence indicator relative to synthetic labels, and disagreement marks a focused inspection set.

E. Qualitative Scenarios

Six hand-crafted dashboard scenarios (Healthy, Recovering, Looping, API Failure, Waiting, Abandoned) support interactive demonstration. They complement but do not replace the quantitative evaluation.

V. Discussion

What works. Waiting and idle-gap features removed a former systematic Waiting failure mode. Dual inference makes uncertainty visible. On the primary run, agreement accuracy is 98.4% on 64% of traces, the 36% disagreement queue contains 75% of rule errors and all LLM-only corrections, and three of four rule Stalled misses are recovered under disagreement.

What does not. Rules are hand-tuned to generators; LLM alone is weaker and systematically maps Abandoned to Stalled ($n=20$). Agreement is not infallible when the LLM path falls back to heuristics. Synthetic evaluation cannot establish real-world robustness. Human inter-rater validation of the state vocabulary remains future work.

Scope of claims. These experiments demonstrate feasibility of hybrid diagnostic inference on controlled synthetic trajectories. They do not claim that hybrid label fusion beats tuned rules on accuracy, nor production generalisation.

VI. Limitations

- Rule thresholds are hand-tuned on the synthetic generators.
- The corpus is synthetic, not drawn from production agents.
- Residual Stalled→Executing errors show brittle thresholds.
- LLM underperforms the tuned rule baseline alone; hybrid value is diagnostic, not LLM dominance.
- Hybrid API reliability: 14/100 judge calls used fallbacks in the primary run.
- No completed human inter-rater κ study is reported in this version.

VII. Conclusion

LLMSheriff investigates hybrid behavioral state inference from agent execution traces, with agreement/disagreement as an interpretable confidence signal. On a controlled synthetic evaluation, rules provide a strong baseline, the LLM contributes complementary signals on uncertain cases, and agreement relative to synthetic labels is strong (98.4% accuracy on the agreement subset in the primary run). The work is exploratory and scoped to the evaluation performed. Future work includes real-world traces, human agreement studies, calibrated learning of thresholds, and broader baselines—improvements that can position the approach for stronger venues after this programme submission.

References

- [1] J. Horkoff et al., “Goal-oriented requirements engineering: An extended systematic mapping study,” *Requirements Engineering*, vol. 21, no. 2, pp. 133–160, 2016.
- [2] A. Dardenne, A. van Lamsweerde, and S. Fickas, “Goal-directed requirements acquisition,” *Science of Computer Programming*, vol. 20, nos. 1–2, pp. 3–50, 1993.
- [3] S. Carberry, “Techniques for plan recognition,” *User Modeling and User-Adapted Interaction*, vol. 11, nos. 1–2, pp. 31–48, 2001.
- [4] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.
- [5] OpenTelemetry Authors, “OpenTelemetry specification v1.26,” <https://opentelemetry.io/docs/specs/otel/>, 2024.
- [6] LangChain, “LangSmith,” <https://www.langchain.com/langsmith>, 2024.
- [7] Langfuse, “Open-source LLM observability,” <https://langfuse.com>, 2024.
- [8] AgentOps, “AI agent observability,” <https://www.agentops.ai>, 2024.
- [9] L. Chen et al., “AI Agent Behavioral Science,” *arXiv:2506.06366*, 2025.
- [10] M. Cherep et al., “A Framework for Studying AI Agent Behavior: Evidence from Consumer Choice Experiments,” in *Proc. ICLR*, 2026. <https://www.media.mit.edu/projects/abxlab/overview/>
- [11] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [12] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. NeurIPS*, 2023.
- [13] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. NeurIPS*, 2023.
- [14] NVIDIA, “Nemotron-3-Ultra-550B,” <https://build.nvidia.com/>, 2025.